DEVELOPERS + AI

WEEK 02 VAULT

Five under-surfaced dev tools that make AI coding agents actually usable — each with a paste-ready prompt.

JUNE 2026 · 5 PICKS · EN / AR

> emdash > code2prompt > ast-grep > LLM > Claude Code Router

curated by Hamzah Al-Ramli · goodnbad.info

Welcome to the Vault.

This week is about the layer most devs never find: the tools that wrap around Claude, Codex, Cursor and Gemini and make them genuinely productive instead of frustrating. Each pick is a real open-source repo — what it does, why it matters for the way you actually work, the GitHub link, and a prompt you drop straight into your AI to set it up.

▸ HOW TO USE EACH PICK

1. Read the hook — decide in 5 seconds if it's for you.
2. Open the GitHub link.
3. Copy the **>_ DROP THIS INTO YOUR AI** prompt into Claude, ChatGPT, or Codex.
4. Let it walk you through setup & first use. No guesswork.

هالأسبوع عن الطبقة اللي أغلب المطورين ما يلقونها: الأدوات اللي تلتف حول Claude و Codex و Cursor و Gemini وتخليهم مفيدين فعلاً بدل ما يطقّشونك. كل أداة ريبو حقيقي مفتوح المصدر — وش تسوي، وليش تهمك بطريقة شغلك، ورابط GitHub، وبرومبت جاهز تنسخه في أي AI ويركّبها لك.

THIS WEEK

AI TOOLS / AGENT ORCHESTRATION · YC W26

emdash

01

★ ~5k · open-source · YC W26 · verdict BOTH

Run five coding agents at once without drowning in terminals. emdash gives each one its own git worktree, so you try multiple fixes in parallel and merge only what works.

شغّل خمس agents يكتبون كود بنفس الوقت بدون ما تغرق بنوافذ ترمينال. emdash يعطي كل واحد git worktree خاص فيه، فتجرّب أكثر من حل بالتوازي وتدمج بس اللي نجح.

WHAT IT IS

emdash is an open-source desktop app for running AI coding agents in parallel, each isolated in its own git worktree and branch so they never step on each other. You assign tasks, watch them run side by side, review the diffs, and merge the winners. It speaks to Claude Code, Codex, Cursor, OpenCode, Gemini, Copilot and more, and can drive remote machines over SSH. You can even pipe issues from Linear, GitHub or Jira straight into an agent.

WHY IT MATTERS FOR YOU

The second you run more than one agent, plain terminals turn into chaos — you lose track of which branch is which and clobber your own work. emdash makes parallel agent coding a normal workflow instead of a juggling act. Still genuinely under-surfaced despite a fast climb to ~5k stars.

repo > github.com/generalaction/emdash

>_ DROP THIS INTO YOUR AI

```
I want to run multiple AI coding agents in parallel without losing track of branches.
Set up emdash (github.com/generalaction/emdash) on my machine: walk me through install,
connecting my Claude Code and Codex CLIs, and creating two isolated worktree tasks for
the same repo so I can compare diffs and merge the better one. Flag any prerequisite I'm
missing first.
```

DEV TOOLS / CLI · RUST

code2prompt

02

★ ~7.4k · Rust · token-aware · verdict BOTH

Stop hand-copying files into your AI. code2prompt turns your whole codebase into one clean, token-counted prompt with the source tree included — in a single command.

بطّل تنسخ الملفات وحدة وحدة للـ AI. code2prompt يحوّل مشروعك كامل لبرومبت واحد نظيف مع شجرة الملفات وعدّاد التوكنز — بأمر واحد.

WHAT IT IS

code2prompt is a fast Rust CLI that ingests a codebase and formats it into a single, well-structured prompt for an LLM. It builds a source-tree view, respects .gitignore, filters by glob patterns, and counts tokens so you stay inside the model's context window. Handlebars templates let you shape the output for code review, bug-finding, or docs, and it has Git integration plus Python bindings for automation and RAG pipelines.

WHY IT MATTERS FOR YOU

Half the pain of AI coding is feeding the model the right context without blowing the token budget. This is the cleanest way to hand a whole repo — or just the relevant slice — to Claude or ChatGPT in one shot. Built in Rust, so it's instant even on large monorepos.

repo > github.com/mufeedvh/code2prompt

>_ DROP THIS INTO YOUR AI

```
I keep manually pasting files into my AI and wasting context. Set up code2prompt (github.com/mufeedvh/code2prompt) on Windows: install it, then show me the exact command to pack only my src/ folder into a single token-counted prompt that includes the source tree and respects .gitignore. Then show me how to use a custom template for a code-review prompt.
```

// PICK 03 STRUCTURAL REFACTOR

DEV TOOLS / CODE SEARCH · RUST

03

ast-grep

★ ~14.5k · Rust · tree-sitter · verdict RUN

grep finds text; ast-grep understands code. Search and rewrite by syntax across thousands of files — the precise refactor tool to hand your AI instead of risky find-and-replace.

grep يلقى نص؛ ast-grep يفهم الكود. ابحث واستبدل حسب التركيب البرمجي عبر آلاف الملفات — أداة الـ refactor الدقيقة اللي تعطيها للـ AI بدل find-and-replace الخطير.

WHAT IT IS

ast-grep is a Rust CLI for structural code search, linting, and rewriting. Instead of matching raw text, you write a pattern in the language itself and it matches the abstract syntax tree — so renaming an API, migrating a deprecated call, or enforcing a rule works correctly even across formatting differences. It supports many languages out of the box via tree-sitter, runs in parallel, and ships an MCP server so an AI agent can run safe, syntax-aware edits at scale.

WHY IT MATTERS FOR YOU

Asking an AI to 'replace every old API call' with plain text matching is how you silently break code. ast-grep gives the agent a surgical, syntax-aware tool, so large refactors and codemods become reliable instead of a gamble. Install from npm, pip, cargo, scoop or homebrew and it just works.

repo > github.com/ast-grep/ast-grep

>_ DROP THIS INTO YOUR AI

```
I need to safely refactor a deprecated function call across my whole codebase without
breaking things. Set up ast-grep (github.com/ast-grep/ast-grep): install it on Windows,
then help me write a structural pattern that finds every call to the old function I'll
name and rewrites it to the new signature. Show me the dry-run preview before applying,
and how to wire its MCP server into Claude Code.
```

AI TOOLS / CLI · PYTHON

LLM

04

★ ~12k · by Simon Willison · plugin ecosystem · verdict BOTH

One command-line tool for every model — cloud or local — with a plugin for nearly everything. Pipe your logs, code, or data straight into an LLM and store every prompt in SQLite.

أداة سطر أوامر واحدة لكل النماذج — سحابية أو محلية — مع plugin لكل شي تقريباً. مرّر اللوقات أو الكود أو الداتا مباشرة للـ LLM، وكل برومبت ينحفظ بـ SQLite.

WHAT IT IS

LLM is a CLI and Python library from Simon Willison for talking to large language models from the terminal. A plugin system connects it to OpenAI, Claude, Gemini, and dozens more — plus fully local models through Ollama and llama.cpp — so you switch backends without changing your workflow. It logs every prompt and response to a local SQLite database, runs interactive chats, generates embeddings, extracts structured data via schemas, and now lets models call tools.

WHY IT MATTERS FOR YOU

It turns any shell pipe into an AI step: pipe a stack trace, a diff, or a CSV straight into a model and get an answer inline. Because everything is logged to SQLite, you build a searchable history of every AI interaction — gold for scripting and audits. Famous in the right circles, still unknown to most devs.

repo > github.com/simonw/llm

>_ DROP THIS INTO YOUR AI

```
Set up Simon Willison's LLM tool (github.com/simonw/llm) on Windows as my terminal AI
layer. Walk me through: install via pip, add a plugin for Claude and one for a local
Ollama model, set my API key, and run a prompt by piping a file into it. Then show me
how to query my logged prompt history from the SQLite database.
```

AI TOOLS / CLAUDE CODE · PROXY

Claude Code Router

05

★ ~26k · MIT · drop-in proxy · verdict RUN

Keep Claude Code's workflow, swap the brain underneath. Route requests to DeepSeek, Gemini, OpenRouter, or a local model — slash your bill and switch models mid-session.

> خلّ شغل Claude Code زي ما هو، وبدّل العقل اللي تحته. وجّه الطلبات لـ DeepSeek أو Gemini أو OpenRouter أو نموذج محلي — قصّ فاتورتك وبدّل النموذج بنص الجلسة.

WHAT IT IS

Claude Code Router is a lightweight proxy that sits between Claude Code and the model provider, intercepting requests and forwarding them to whatever backend you choose — DeepSeek, Gemini, OpenRouter, Ollama, and more. You can set rules so cheap or local models handle simple steps while a premium model takes the hard ones, and switch providers mid-session. It keeps the Claude Code interface you already know while breaking the vendor lock-in underneath.

WHY IT MATTERS FOR YOU

If you live in Claude Code, your single biggest lever is the model bill — and routing simple work to cheaper models can cut it dramatically without changing how you work. It's also your escape hatch: rate-limited or a model goes down, you reroute in seconds. The practical cost layer most heavy users eventually need.

repo > github.com/musistudio/claude-code-router

>_ DROP THIS INTO YOUR AI

```
I want to cut my Claude Code costs without changing my workflow. Set up Claude Code
Router (github.com/musistudio/claude-code-router): install it, configure it as a proxy
in front of Claude Code, add one cheap provider (DeepSeek or OpenRouter) and a local
Ollama fallback, and write a routing rule that sends simple tasks to the cheap model and
hard ones to the premium model. Show me how to verify it's routing correctly.
```

That's the Vault for this week.

Five tools, five prompts. Wire even one into your workflow this week and your agents stop fighting you. Real infrastructure you run, not another thread to bookmark.

Next week: more under-surfaced dev & AI picks — local model stacks, agent memory layers, and the self-hosting tools nobody talks about.

Found one of these useful? Forward this to one person who'd get it. That's how the Vault grows.

خمس أدوات، خمس برومبتات. ركّب ولو وحدة بسير شغلك هالأسبوع وبتلاحظ الـ agents وقفوا يعاندونك. بنية تحتية حقيقية تشغّلها، مش ثريد ثاني تحفظه وتنساه. عجبتك وحدة؟ ابعثها لمطور تعرفه — هكذا تكبر الخزينة.